

4 EXPERIMENTS

We evaluate computer-use agents across multiple models and observation spaces on our *HackWorld* benchmark, analyzing both task completion rates and tool usage patterns to understand fundamental limitations in cybersecurity reasoning capabilities. The prompt can be found in Section D.

4.1 EXPERIMENTAL SETTINGS

CUAs. We employ two types of agents to construct computer-using agents, including four top-performing proprietary models in general CUA tasks (i.e., [Claude-3.5-Sonnet \(2024\)](#), [Claude-3.7-Sonnet \(2025\)](#), [Claude-4-Sonnet \(2025\)](#), [Claude-4-Opus \(2025\)](#)) and two open-source GUI action model, i.e., [UI-TARS-1.5-7B \(Qin et al., 2025\)](#) and [Qwen-2.5-VL \(Bai et al., 2025\)](#). In our experiments, all models are deployed using a server equipped with A100 80GB GPUs with vLLM. The Kali virtual machine is run on a bare metal AWS instance. More details are as in Section B.1.

Observation Space. The observation space defines the environmental information accessible to an agent. We adopt the following three well-established configurations: (1) *Screenshot* ([Xie et al., 2024](#)): Following OSWorld ([Xie et al., 2024](#)), we capture a screenshot of the entire computer screen. For screen resolution, we set a default value of 1280x720, and it also offers a 16:9 aspect ratio. (2) *Screenshot + a11ytree* ([Chromium; Wang et al., 2021](#)): The a11ytree is a structured, text-based representation of the interface’s semantic structure, devoid of visual data, designed for agents that process purely textual input. To further enhance the action execution capabilities of computer-using agents, especially for models with weaker grounding abilities, we utilize a combined input of screenshots and a11ytree. (3) *Set-of-Marks* ([Yang et al., 2023a](#)): It’s a visual prompting paradigm that segments an image into discrete, marked regions to augment an agent’s visual grounding capabilities.

4.2 RESULT ANALYSIS

In this section, we evaluate computer-use agents on *HackWorld*, analyzing task completion rates and tool usage patterns across multiple models and observation spaces.

4.2.1 OVERALL PERFORMANCE EVALUATION

Table 2: Success rates of computer-use agents across different observation spaces.

Observation	Screenshot	Screenshot + a11ytree	Set-of-Marks
Claude-3.5-Sonnet	2.78%	5.56%	2.78%
Claude-3.7-Sonnet	11.11%	8.33%	11.11%
Claude-4-Sonnet	0.00%	0.00%	0.00%
Claude-4-Opus	5.56%	5.56%	2.78%
UI-TARS-1.5-7B	0.00%	0.00%	0.00%
Qwen-2.5-VL-72B-Instruct	0.00%	0.00%	0.00%

[Table 2](#) shows the overall performance of the five evaluated computer-use agents (i.e., Claude-3.5-Sonnet, Claude-3.7-Sonnet, Claude-4-Sonnet, Claude-4-Opus, UI-TARS-1.5-7B, and Qwen-2.5-VL-72B-Instruct), measured by their success rate in solving 36 distinct cybersecurity challenges. Detailed results of each agent across different observation spaces can be found in Section B.2.1.

***HackWorld* poses substantial challenges, and more recent models do not necessarily yield better outcomes.** From a quantitative perspective, the results reveal substantial variation in performance. Claude-3.7-Sonnet achieves the highest average success rate (10.18% across all observation spaces), which is nearly double that of Claude-4-Opus (4.63%) and over three times that of Claude-3.5-Sonnet (3.71%). By contrast, UI-TARS-1.5-7B and Qwen-2.5-VL-72B-Instruct exhibit a 0% completion rate in all or most conditions, highlighting severe limitations in their ability to engage with complex tasks. Importantly, the superior performance of Claude-3.7-Sonnet over later Claude-4 models brings into question the prevalent assumption that model size and recency guarantee higher task competence.

The ability of agents to control the computer is not the main bottleneck. Analyzing the impact of observation spaces, the screenshot condition yields the most consistent performance, with an average success rate of 3.89% across models. The screenshot + alltree setting provides a modest improvement for some models (e.g., Claude-3.5-Sonnet), but the overall gain is limited (mean success rate 3.97%). In contrast, the Set-of-Marks representation performs worst (mean success rate 3.17%), suggesting that abstract symbolic encodings may lose critical contextual cues necessary for effective task execution. However, a one-way ANOVA test (Quirk, 2012) across observation spaces reveals that the difference in success rates is not statistically significant ($p > 0.1$), reinforcing the argument that perceptual fidelity is not the primary bottleneck in cybersecurity task execution. This suggests that future CUAs should prioritize enhancing environment exploration, reasoning over feedback, and the integration of cybersecurity domain knowledge. In summary, it suggests that the upper performance limit for cybersecurity tasks is primarily limited by reasoning, planning, and tool orchestration capabilities rather than perceptual input.

HackWorld stimulates natural inference-time scaling through exploration. As shown in Table 4, the best-performing CUA in our setting (Claude 3.7 Sonnet) solves more tasks (+5.6%) when allowed additional steps. However, we found the underlying mechanism fundamentally different from prior CUA benchmarks (Xie et al., 2024; Bonatti et al., 2024; Rawles et al., 2024), which typically encourage agents to follow a fixed, canonical long trajectory to complete tasks. In contrast, **HackWorld has no predefined solution path**; instead, it requires CUAs to actively explore the environment, gather relevant information, and iteratively test their hypotheses. Once sufficient evidence is collected, the flag can often be retrieved in just a few decisive steps.

4.2.2 TOOL USAGE ANALYSIS

Table 3: Analysis of Tool Usage by Observation Method and Model. **% Used**: percentage of trajectories using at least one tool. **Avg**: average tools per trajectory. **Avg⁺**: average tools per trajectory for active users only (excluding zero-tool cases). **Top 3 Tools**: most frequently used tools. More Detailed tool usage results can be found in Section B.2.2

Observation	Model	% Used	Avg	Avg ⁺	Top 3 Tools
Screenshot	Claude-4-Sonnet	44.44	0.97	2.19	dirb, DirBuster, Burp Suite
	Claude-3.7-Sonnet	58.33	2.33	4.00	dirb, Nikto, WhatWeb
	Claude-4-Opus	44.44	0.86	1.94	dirb, DirBuster
	Claude-3.5-Sonnet	88.89	5.33	6.00	dirb, Nikto, DirBuster
Screenshot +alltree	Claude-4-Sonnet	38.89	0.86	2.21	dirb, DirBuster, WhatWeb
	Claude-3.7-Sonnet	72.22	2.14	2.96	dirb, DirBuster, Nikto
	Claude-4-Opus	38.89	0.72	1.86	dirb, DirBuster, Netcat
	Claude-3.5-Sonnet	94.44	4.28	4.53	dirb, DirBuster, Nikto
Set-of-Marks	Claude-4-Sonnet	16.67	0.33	2.00	dirb, DirBuster
	Claude-3.7-Sonnet	69.44	2.08	3.00	dirb, DirBuster, Nikto
	Claude-4-Opus	19.44	0.36	1.86	dirb, DirBuster, Nikto
	Claude-3.5-Sonnet	91.67	4.28	4.67	dirb, DirBuster, Nikto

Table 4: Successful rate (%) of Models across different step limits.

Model	15 steps	50 steps	100 steps
Claude 3.7 Sonnet	11.1	11.1	16.7
UI-TARS-7B	0.0	0.0	0.0

Tool usage efficiency. Analysis of the data indicates that frequent tool invocation does not necessarily imply high engagement efficiency. For example, Claude-3.5-Sonnet invoked tools in nearly all trajectories (88.89–94.44%) and averaged 4–6 tool calls per trajectory, while other models achieved similar or better engagement with fewer calls. This demonstrates that efficiency and selectivity in tool usage, rather than raw frequency, are critical characteristics of effective tool integration.

Impact of observation space. Across models, differences in tool usage patterns between observation spaces are relatively modest, indicating limited sensitivity to the format of perceptual input. For instance, Claude-3.7-Sonnet shows comparable tool invocation behavior across *Screenshot*, *Screenshot + a11yTree*, and *Set-of-Marks* observation spaces. Similarly, Claude-4-Opus maintains similar usage levels across all observation formats. It suggests that enhancing the structure or symbolic content of the observation space provides limited additional benefit once basic perceptual fidelity is ensured.

Inter-model contrasts. Variations in tool usage are driven not by model scale or observation space, but by model-specific strategies. This is evidenced by the more selective tool use of smaller or earlier models compared to larger, recent ones.

Key Insights. Overall, the analysis highlights three main observations: (1) efficient and selective tool usage is more informative than sheer frequency of tool calls, emphasizing the importance of strategic tool integration; (2) once basic perceptual fidelity is ensured, additional structuring of the observation space (e.g., *a11yTree*, *Set-of-Marks*) provides limited benefit; and (3) differences between models are more pronounced than differences between observation spaces, showing that reasoning strategy and model-specific behavior dominate tool usage patterns.

5 RELATED WORK

Computer-Use Agents Computer-use agents (CUAs) are AI systems capable of interacting with digital interfaces through human-like actions such as clicking, typing, and navigation. Recent work has pushed visual grounding for action modeling for generic GUI control: OS-ATLAS (Wu et al., 2024b) builds a cross-platform (desktop/web/mobile) foundation action model with large-scale GUI grounding data and demonstrates strong gains across multiple benchmarks, while SeeClick (Cheng et al., 2024) shows that pretraining on GUI grounding from screenshots substantially improves downstream GUI automation. Beyond DOM-aware approaches, Aguis (Xu et al., 2024b) proposes a pure-vision GUI agent with a unified action space that generalizes across platforms and apps. CUA progress also benefits from scalable trajectories/data: OS-Genesis (Sun et al., 2024) introduces reverse task synthesis to automatically construct high-quality GUI trajectories without predefined tasks, and AgentTrek (Xu et al., 2024a) scales web-agent trajectories via guided replay with public tutorials, yielding measurable improvements when training or prompting agents. From a system/platform perspective, OS-Copilot (Wu et al., 2024a) presents a self-improving, cross-application computer agent spanning web, terminal, files, and office tools, and reports steady skill accumulation on realistic tasks, while OpenCUA explores a systematic framework for scaling CUA annotations. On training paradigms, Learn-by-Interact (Su et al., 2025) offers a data-centric adaptation pipeline that synthesizes interaction trajectories and backward-constructed instructions and reports consistent gains on standard CUA benchmarks. In parallel, UI-TARS-2 (Wang et al., 2025) scales multi-turn RL for GUI-centered agents. Together, these advances complement existing benchmarks (e.g., WebShop (Yao et al., 2022), MiniWoB++ (Shi et al., 2017), Mind2Web (Deng et al., 2023), OSWorld (Xie et al., 2024)) by strengthening perception–action coupling, scaling data/trajectories, and improving systems and training recipes for general-purpose CUAs.

Current evaluations largely ignore security considerations (Evtimov et al., 2025; Mudryi et al., 2025; Zhang et al., 2024). Although CUAs can complete tasks effectively, their behavior in risky scenarios, such as interacting with phishing content or handling sensitive data, remains underexplored. To address this limitation, *HackWorld* introduces a benchmark specifically designed to evaluate the cybersecurity capabilities and vulnerabilities of CUAs by integrating security challenges within authentic computer-use contexts.

Benchmarking Cybersecurity Capabilities Robust benchmarks are essential for evaluating the cybersecurity abilities of CUAs. Prior approaches can be grouped into static question-answering, automated single-step exploitation, and interactive agent-based evaluation. Multiple-choice datasets probe basic cybersecurity knowledge (Li et al., 2024; Tihanyi et al., 2024; Liu, 2023), but provide

limited insight into operational behavior and are sensitive to prompt formulation (Qi et al., 2025; Łucki et al., 2025). Single-step frameworks, such as AutoAdvExBench (Carlini et al., 2025) and CyberSecEval (Bhatt et al., 2023), assess autonomous exploitation of adversarial defenses or code snippets but do not capture the extended, adaptive sequences required in real-world attacks. Interactive, agent-based frameworks that incorporate tool usage better approximate realistic conditions. Capture-the-flag (CTF) environments require multi-step reconnaissance, exploitation, and access maintenance, closely reflecting authentic attacker workflows. Recent frameworks (Abramovich et al., 2025; Mayoral-Vilches et al., 2025; Zhuo et al., 2025a;b) combine rich simulations with structured attack-chain analysis, evaluating the complete process of exploitation rather than isolated actions. *HackWorld* builds on this interactive paradigm by providing a structured, multi-step environment that simulates realistic web attack chains. Unlike specialized penetration testing benchmarks, *HackWorld* uniquely evaluates general-purpose agents capabilities in realistic web security scenarios.

Operational Security Evaluation A central challenge in evaluating interactive agents is simulating multi-stage attacks and detecting successful exploits. Frameworks, e.g., AI kill-chain (Rodriguez et al., 2025) and Agent Security Bench (Zhang et al., 2024) formalize this goal. Platforms like Pen-testGPT (Deng et al., 2024) and EnIGMA (Abramovich et al., 2024) operationalize it by immersing agents in penetration testing, showing that better tool use mitigates deficits in multi-step reasoning. Complementing this, benchmarks like WASP (Evtimov et al., 2025) focus on explicit detection of exploits, while frameworks like PenHeal (Huang & Zhu, 2023) extend evaluation to defensive remediation. These works collectively establish the principles of end-to-end attack simulation with integrated detection, principles that directly inform *HackWorld*’s design.

6 DISCUSSION AND FUTURE WORK

The findings of this study provide several important insights for the development of LLM-based agents in web cybersecurity tasks, highlighting both current limitations and future directions.

Common failure patterns. Our evaluation of agent performance across various observation spaces reveals several systematic failure patterns that warrant further examination. These recurring shortcomings not only highlight the current limitations in computer-use agents but also point to critical areas for improvement in future agent design. Specifically, we identify eight predominant failure modes as follows: (1) *Ineffective tool selection and output parsing.* Agents frequently executed multiple steps to launch different or identical tools, without adequately analyzing prior outputs. Clues such as `robots.txt` entries or repository artifacts were often detected but not utilized. It was common for agents to read content without subsequent action, and upon encountering tool errors, they tended to abandon diagnostic efforts in favor of switching tools arbitrarily. (2) *Poor failure recovery and plan repair.* When faced with routine errors (e.g., HTTP 404, 403, or 302 responses), agents typically stall or proceed without correcting fundamental issues such as base paths, authentication state, or request parameters. Their request patterns remained narrow, with limited variation in headers, methods, or encodings. (3) *Gaps in directory and source enumeration.* Agents either omitted systematic enumeration (using tools such as `dirb`, `DirBuster`, or `gobuster`) or failed to persist enumeration results for deeper investigation, thereby breaking the chain of evidence. (4) *Incomplete port/service mapping.* `nmap` runs often lacked `-p/service` versioning, leading to partial service pictu and missed attack surfaces. (5) *Lack of authentication bypass/session management.* Agents failed to establish/maintain sessions (cookies, CSRF), did not attempt standard bypasses (weak creds, SQLi login, password reset, JWT tamper, IDOR, Host/Origin spoof), or failed to reuse session context later. (6) *Misclassification of service types.* For instance, port 6080 is frequently misinterpreted as native VNC, leading agents to launch RFB clients rather than treating the service as noVNC. (7) *Superficial SQL injection testing.* Agents attempt UNION-based attacks or employ `sqlmap` without performing differential response analysis or establishing clear criteria for successful exploitation, rendering probes ineffective. (8) *Knowledge-driven dead loops.* When uncertain, agents often get stuck in loops of repetitive, ineffective actions, creating activity without progress.

From perception to strategy. Perception methods did not unlock progress: neither SOM nor ally-tree consistently raised success. Agents could “read” pages and tool outputs but failed to aggregate clues (e.g., `robots.txt`, `exposed.git`, differential HTTP codes) into an exploit plan or a credential flow. In contrast, Claude-3.7 succeeded more often by selectively analyzing key clues and reusing them, while keeping tool usage focused rather than exhaustive. Our analysis indicates

that perceptual fidelity, such as parsing UI layouts, textual outputs, and symbolic representations, is no longer the primary bottleneck in web cybersecurity tasks. Agents were generally able to extract relevant information from the environment, but often failed to synthesize intermediate results into coherent and actionable strategies. Future work should prioritize strategic reasoning and decision-making over better perception, as enhancing multi-step planning will yield greater gains.

Challenging the scaling hypothesis. Model strength does not necessarily result in higher success. Despite being larger/newer, Claude-4-Opus underperformed across methods, while Claude-3.7-Sonnet achieved the best overall success. This challenges a naive scaling hypothesis for web-security tasks: performance hinged more on planning discipline and strategy control than raw model capacity. This observation aligns with recent evidence questioning the monotonic scaling hypothesis in complex reasoning tasks (Kaplan et al., 2020; Wei et al., 2022). The results show we need evaluations that measure reasoning and strategic decision-making, not just benchmark accuracy.

Lack of Strategic Tool Use. More tool calls are not equal to better outcomes. Agents frequently cycled through scanners (e.g., dirb, Nikto, Wfuzz) with near-duplicate parameters, or switched tools immediately after a minor error, instead of diagnosing and repairing the current step. Claude-3.5-Sonnet made the most tool calls in SOM but had low success, showing it knows when evidence is needed but lacks an effective loop to act on it.

Implications for Tool/Interface Design. The failures in our evaluation reflect a mismatch between human-oriented tool UX and agent-oriented needs. Current CLI tools are verbose, loosely structured, and error-opaque, which forces LLMs to guess. Adopting Agent eXperience (AX) principles would directly target the above failure modes: machine-readable outputs (JSON/JSONL), explicit state and error codes, persistent session/context hooks, and asynchronous progress for long tasks. Standardized wrappers (e.g., MCP/Arazzo-style contracts) can expose what the tool needs, what it returns, and how to proceed next, enabling step-by-step reasoning loops. Practically, port scanning and fingerprinting should emit canonical fields (protocol, open ports, service/version, confidence, evidence snippets) so agents can carry forward results into path testing, auth flows, and exploit crafting, rather than restarting yet another tool.

7 CONCLUSION

This paper proposes *HackWorld*, a benchmark for the systematic evaluation of Cybersecurity Agents (CUAs) in exploiting web vulnerabilities. Experimental results reveal severe limitations in current agents’ capabilities: even state-of-the-art models achieved low success rates, with the top performer solving merely 11.1% of tasks. Our analysis identifies the core bottleneck not as perceptual understanding, but as a critical deficit in strategic reasoning and tool orchestration for vulnerability discovery and exploitation. These findings underscore a significant need for improvement in CUA cybersecurity skills and outline key research directions. *HackWorld* thus establishes a vital foundation for developing autonomous agents with advanced penetration testing capabilities.

ETHICS AND REPRODUCIBILITY STATEMENT

The development and release of cybersecurity evaluation frameworks inherently involves dual-use considerations, as these technologies can advance both defensive security research and potentially enable malicious applications (Rad, 2015). We acknowledge that *HackWorld* and the evaluated CUAs possess dual-use characteristics that warrant careful ethical consideration.

Our framework systematically evaluates CUAs’ capabilities in web vulnerability exploitation, which could potentially inform the development of both defensive and offensive cybersecurity tools. While current agents achieve relatively low success rates (11.1% best performance), the rapid advancement of models suggests future iterations may demonstrate significantly enhanced capabilities (Guo et al., 2025; Jones et al., 2025). Our benchmark provides methodology for evaluating agents on vulnerability discovery and exploitation, which represents a dual-use technology.

We believe the benefits of public release outweigh potential risks for several reasons. First, understanding current CUA capabilities is essential for both defensive security research and informed policy decisions about AI in cybersecurity contexts (Kapoor et al., 2024; Guha et al., 2024). Second, similar frameworks have already been released for cybersecurity evaluation (Deng et al., 2024; Happe